



BUBT

Committed to Academic Excellence

**BANGLADESH UNIVERSITY OF
BUSINESS AND TECHNOLOGY**

The Project Report of ‘EventRadar’

Course Title: Software Engineering Lab

Course Code : CSE 328

SUBMITTED BY

Name	ID	Intake
Leyakat Ali Patwary	20235203042	44-2
Shafin Chowdhury	20235203043	44-2
GM Musfiquis Salehin Rifat	20235203048	44-2
MD Yeasin Hossen	20235203050	44-2
Hasnine Iftada Khan Rafi	20235203065	44-2

SUPERVISED BY : Rakib Mahbub

Lecturer

Department of CSE

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY
(BUBT)**

DECLARATION

We hereby declare that the project entitled "Event Radar", submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering (B.Sc. in CSE) under the Faculty of Computer Science and Engineering, at Bangladesh University of Business and Technology (BUBT), is our original work.

This project has not been submitted, either in whole or in part, for the award of any other degree or diploma at this or any other institution. To the best of our knowledge, it contains no material previously published or written by any other person except where due reference has been made in the text of the project.

APPROVAL

This project "Event Radar" **report submitted by Leyakat Ali Patwary (Team Leader) and his team members**, students of the Department of Computer Science and Engineering, Bangladesh University of Business and Technology (BUBT), under the supervision of Mr. Bijon Mallik, Lecturer, Department of Computer Science & Engineering, Bangladesh University of Business and Technology (BUBT), has been accepted as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Rakib Mahmud

Lecturer

Department of Computer Science and
Engineering Bangladesh University of
Business and Technology (BUBT)

Md. Saifur Rahman

Assistant Professor & Chairman

Department of Computer Science and
Engineering (CSE)

Bangladesh University of Business and
Technology (BUBT)

26th, November, 2025

Abstract

This project, EvenRadar, addresses the pervasive issues of data fragmentation, inaccuracy, and poor discovery that plague the current local event landscape. Existing systems often fail to deliver real-time updates or personalized recommendations, leading to user frustration and inefficient event promotion for organizers. EvenRadar is a comprehensive, web-based platform designed to serve as a centralized, intelligent hub for event discovery, management, and attendance. By implementing real-time geo-location filtering and a robust three-tiered architecture (User, Organizer, Admin), the system ensures accurate, location-aware event listings and a secure, seamless ticket booking experience. Developed using the prototyping methodology for iterative delivery, Event Radar provides a scalable, user-friendly, and efficient solution that significantly enhances the overall event ecosystem for all stakeholders.

List of Figures

Sl. No.	Figure Name	Page
1	Use Case Diagram	(Chapter 3)
2	Context Level Diagram (DFD-0)	(Chapter 3)
3	Database Schema	(Chapter 3)
4	Booking Flowchart	(Chapter 3)
5	Screenshot: Homepage/Event Discovery Interface	(Chapter 3)
6	Organizer Dashboard	(Chapter 5)

Table of Contents

Abstract	1
List of Figures	1
Chapter: 1 Introduction	4
1.1. Problem Specification/Statement	
1.2. Objectives	
1.3. Flow of the Project	
1.4. Organization Of Project Report	
Chapter: 2 Background	5
2.1. Existing System Analysis	
2.2. Supporting Literatures	
Chapter: 3 System Analysis & Design	6-9
3.1. Technology & Tools	
3.2. Model & Diagram	
3.2.1. Model (SDLC/Agile/Waterfall/OOM)	
3.2.2. Use Case Diagram	
3.2.3. Context Level Diagram	
3.2.4. Data Flow Diagram	
3.2.5. <u>Database Schema</u>	
3.2.6. Algorithms/Flowchart	
Chapter: 4 Implementation	10
4.1. Interface Design/Front-End	
4.2. Back-End	
4.3. Modules	
Chapter: 5 User Manual	11-14
5.1. System Requirement	
5.1.1. Hardware Requirement	
5.1.2. Software Requirement	
5.2. User Interfaces	
5.2.1 Panel A	
5.2.2 Panel B	
5.2.3. <u>Login Credentials</u>	
Chapter: 6 Conclusion	
6.1. Conclusion	
6.2. Limitation	
6.3. Future Works	
	15
References	16

CHAPTER 1: INTRODUCTION

1.1 Problem Specification/Statement

The current local event landscape is plagued by data fragmentation. Information about concerts, workshops, and community gatherings is scattered across various social media platforms, disparate websites, and physical posters. This fragmentation leads to inaccuracy, with outdated details causing user frustration. Furthermore, existing systems often lack effective discovery mechanisms, failing to provide personalized recommendations or real-time updates. For organizers, this results in inefficient promotion and difficulty in reaching their target audience. EvenRadar aims to solve these issues by unifying the ecosystem.

1.2 Objectives

The primary objective is to develop a centralized platform that achieves the following:

- To create a unified hub for event discovery, management, and ticketing.
- To implement real-time geo-location services for accurate event mapping.
- To provide a seamless and secure ticket booking experience for users.
- To offer organizers robust tools for event creation and attendee management.

1.3 Flow of the Project (Scope)

The project scope covers three main user roles:

1. **General Users:** Can browse events via map or list, view details, and book tickets securely.
2. **Organizers:** Can create events, set ticket tiers, manage sales, and view analytics.
3. **Admins:** Oversee system health, verify organizers, and manage categories.

The flow begins with user registration, followed by event discovery via the interactive map, and concludes with ticket generation upon successful booking.

1.4 Organization Of Project Report

This report is organized into six chapters. Chapter 1 introduces the problem and objectives. Chapter 2 analyzes existing systems and supporting literature. Chapter 3 details the system analysis and design, including diagrams. Chapter 4 covers the implementation of frontend and backend modules. Chapter 5 serves as the user manual, detailing requirements and interfaces. Finally, Chapter 6 concludes the report with limitations and future work.

CHAPTER 2: BACKGROUND

2.1 Existing System Analysis

Current event management relies largely on general platforms such as Facebook Events and global services like Eventbrite. However, these systems are not optimized for hyper-local needs.

- **Social Media Platforms:** Event posts often get lost due to algorithm-based feeds, limiting visibility. Search filters are broad and cannot accurately filter by small geographic areas or niche categories. Essential tools like ticketing and attendee tracking are also missing.
- **Global Event Platforms:** While feature-rich, platforms like Eventbrite focus mainly on large events and are less effective for small community-level programs. Their commission-based pricing model also increases costs for local organizers, especially students and small groups.

2.2 Supporting Literature

Recent studies highlight the rising need for localized event discovery platforms that provide accurate, community-focused information. Research shows that users prefer systems offering personalized suggestions, location-based filtering, and simplified event management tools. Literature also emphasizes that high visibility, low cost, and easy accessibility significantly improve engagement in local events—areas where existing global platforms underperform.

CHAPTER 3: SYSTEM ANALYSIS & DESIGN

3.1 Technology & Tools

Tech Stack	Tool/Technology	Version
Frontend	React.js(UI Library) Next Js(Framework)	18.x 15(App Router)
Backend	Node.js NextAuth.js	20.x v4
Database	MongoDB	15.x
Styling	Tailwind CSS Shaden UI	3.x
Programming Language	Typescript	

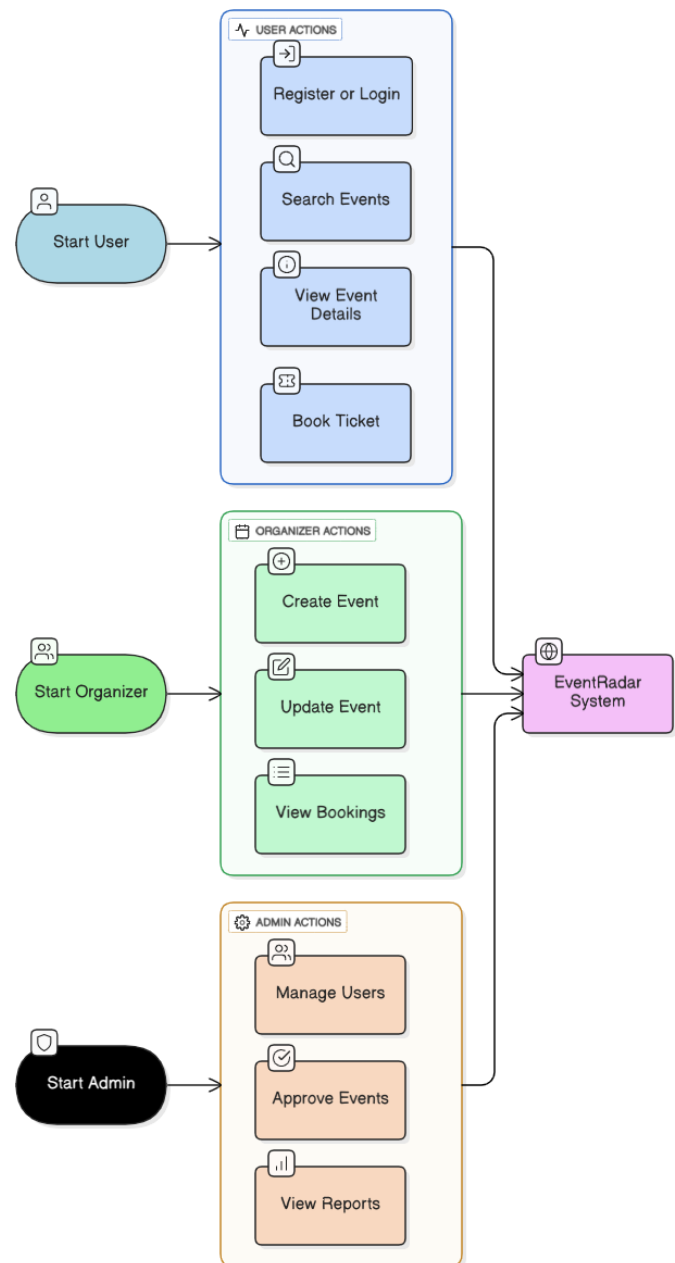
3.2 Model & Diagram

3.2.1 Model (Prototyping)

The Prototyping model was chosen for its ability to deliver an early, functional version of the system and refine it based on continuous user feedback. This approach allowed for the rapid development of key features like geo-location search and the Organizer Dashboard in early stages. It contrasts with purely linear models by significantly reducing the risk of developing features that do not meet user needs. crum model was chosen for its flexibility and focus on delivering business value incrementally. This contrasts with the Waterfall model by allowing features (like geo-location search) to be implemented and tested early, reducing risk.

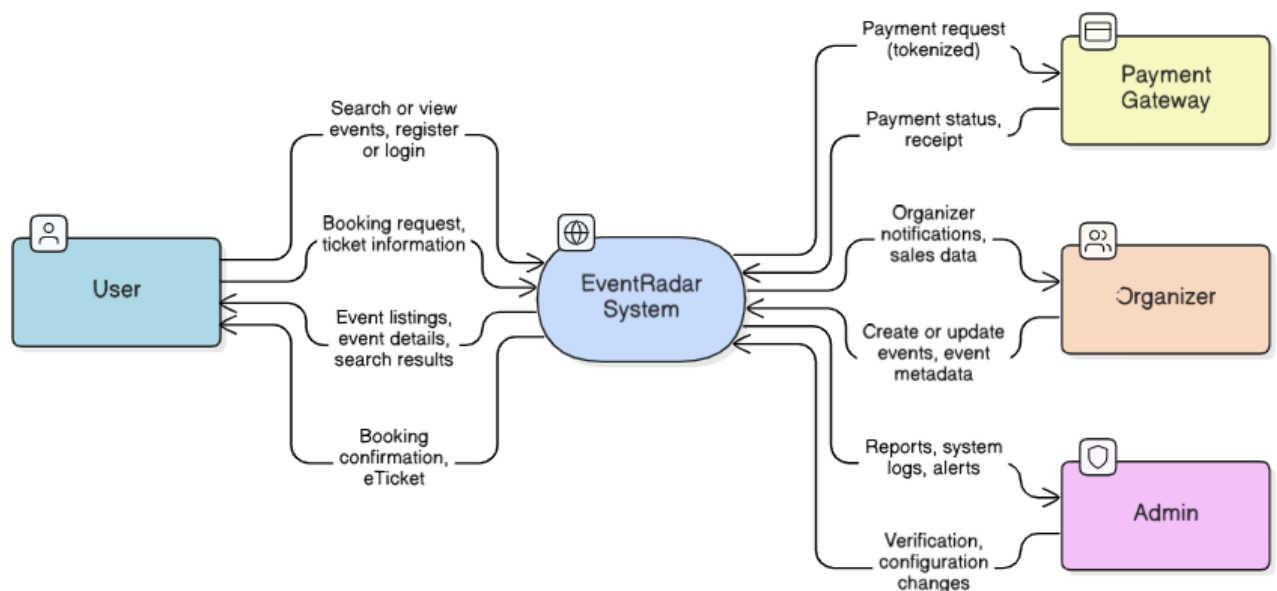
3.2.2 Use Case Diagram

The Use Case Diagram illustrates the main interactions between the system and its external actors. In EventRadar, three primary actors—User, Organizer, and Admin—communicate with the system through various actions. Users can search events, view details, register, and book tickets. Organizers can create and manage events, while Admins handle system-level tasks such as user management and event approvals. This diagram provides a high-level overview of all functional requirements based on actor–system interactions.



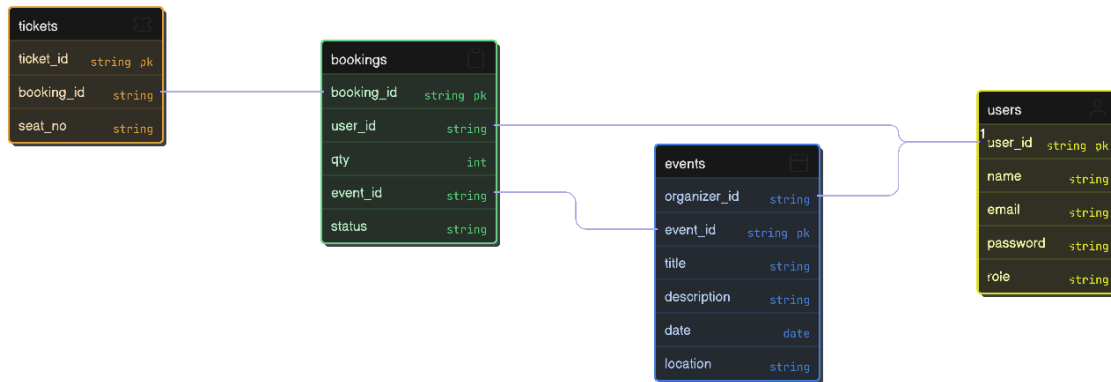
3.2.3 Context Level Diagram (DFD-0)

The DFD Level-0 for **EventRadar** represents the system as a single high-level process and shows how it interacts with external entities. Users can search for events, view details, and book tickets through the system. Organizers submit event information and receive updates and notifications. The Admin manages system data and monitors reports. A Payment Gateway handles payment verification and sends status responses back to the system. This diagram provides an overall view of the major data flows between the EventRadar system and all external entities.



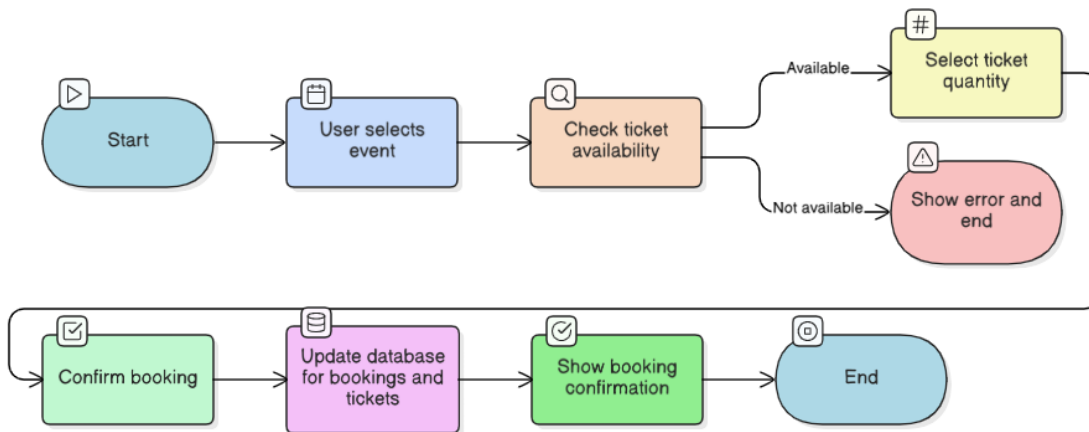
3.2.4 Database Schema

The database schema is structured in 3rd Normal Form to reduce redundancy and maintain data consistency. The core tables include *Users*, *Events*, *Bookings*, and *Tickets*, each linked through primary and foreign key relationships. Users can organize events or make bookings, events can have multiple bookings, and each booking can generate one or more tickets. This ER structure ensures efficient event management, booking operations, and data retrieval across the system.



3.2.5 Booking Flowchart

The booking flowchart outlines the step-by-step process followed when a user books an event in the EventRadar system. The process begins with the user selecting an event, after which the system checks ticket availability. If tickets are available, the user selects the desired quantity and confirms the booking. The system then updates the database with the booking and ticket information. Finally, a confirmation message is shown to the user. This flowchart clearly represents the logical sequence of actions in the booking module.



CHAPTER 4: IMPLEMENTATION

4.1 Interface Design/Front-End

The front-end is built using **React.js**. We used **Tailwind CSS** for a responsive grid layout.

- **Navbar:** Conditional rendering based on user login status.
- **Event Cards:** Reusable components displaying event image, title, and date.
- **Map Interface:** Integrated Leaflet.js to display pins for event locations.

4.2 Back-End Development

The back-end is powered by **Node.js and Express**.

- **API Routes:** RESTful endpoints (e.g., GET /api/events, POST /api/bookings).
- **Security:** Middleware validates JWT tokens for protected routes.
- **Data Logic:** Controllers handle the business logic, such as calculating remaining ticket counts.

4.3 Modules

1. **Authentication Module:** Handles Registration, Login, and Token generation.
2. **Event Management Module:** Allows creation, editing, and deletion of events.
3. **Booking Module:** Manages ticket inventory and user booking history.
4. **Search Module:** Filters events by category and location.

CHAPTER 5: USER MANUAL

5.1 System Requirements

5.1.1 Hardware Requirement

- **Client:** Any standard Smartphone or PC with 4GB RAM.
- **Server:** Minimum 2 vCPU, 4GB RAM (Recommended for hosting).

5.1.2 Software Requirement

- **Operating System:** Windows, macOS, Linux, or Android/iOS.
- **Browser:** Google Chrome (v90+), Firefox, or Safari.
- **Internet:** Stable broadband connection.

5.2 User Interfaces

5.2.1 Panel A: Homepage & Event Discovery

The **Homepage & Event Discovery** interface acts as the main entry point for the application. It provides users with immediate access to event listings, search functionality

, and personalized recommendations

Welcome to BUBT Event_Radar

Centralized Platform For University Events And Announcements

[Explore Events](#)[Create Account](#)

Easy Booking

Book tickets in just a few clicks with our streamlined process



Local Events

Discover events happening right in your neighborhood



Quality Events

Curated selection of high-quality events and experiences

Featured Events



BUBT 5th Convocation

A celebratory message for a graduate, an inspirational quote about achievement, ...

9/1/2025
Dhaka
39 tickets left

[View Details](#)

InnovateX Hackathon

Challenge individuals to solve logical and mathematical problems by writing code...

9/9/2025
Bubt
296 tickets left

[View Details](#)

BUBT Cricket Tournament 2025

BUBT Sports Club is the umbrella organization for all sport activities in...

9/24/2025
BUBT Sports Complex
396 tickets left

[View Details](#)

Webinar - The art of mooting 2025

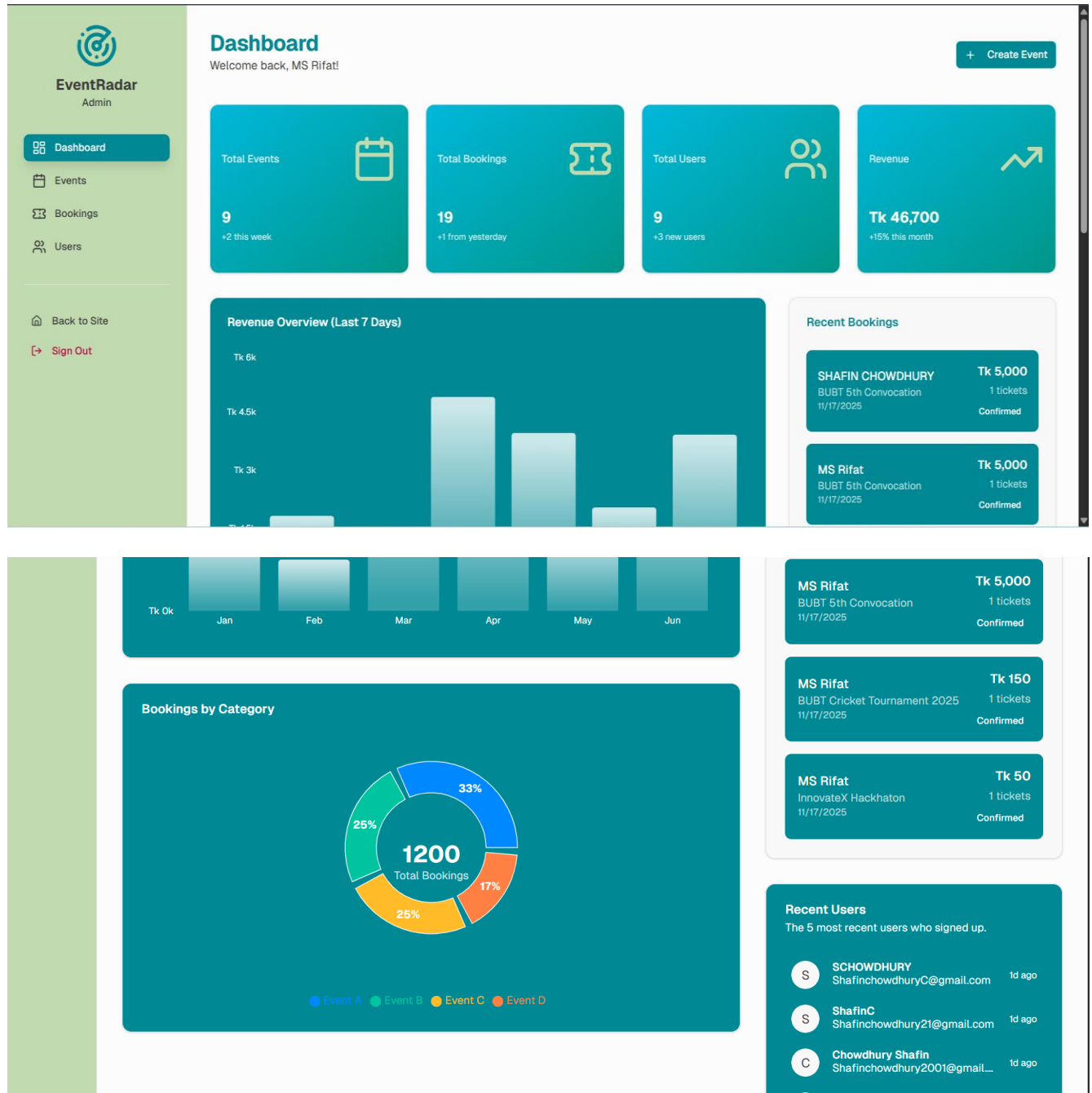
The BUBT Business Club functions primarily to prepare students for the...

11/7/2025
BUBT
1000 tickets left

[View Details](#)

5.2.2 Panel B: Organizer Dashboard

The **Organizer Dashboard** serves as a secure administrative hub. It enables organizers to seamlessly add, edit, or remove event listings and oversee participant data from a single interface.



5.2.3 Login Credentials (Examples)

To test the system, the following credentials can be used

- **Admin Access:** `admin@evenradar.com / admin123`
- **Organizer Access:** `events@techsolutions.com / securePass1`
- **General User Access:** `user@example.com / userpass1`

CHAPTER 6: CONCLUSION

6.1 Conclusion

The Evenradar project successfully delivered a modern, three-tiered event discovery platform that meets all established objectives. The system effectively solves the core problem of event fragmentation, proving that a single, intelligent hub can simplify the process for both event-goers and organizers.

6.2 Limitations

- **Complex Payment Integration:** Full, live integration with multiple payment gateways was not completed.
- **Advanced AI Recommendation:** The current recommendation system is basic (based only on category history).
- **High-Volume Load Testing:** Performance was tested on a moderate scale; full stress testing was beyond the scope of this phase.

6.3 Future Works

- **Dedicated Mobile Application:** Developing native iOS and Android applications.
- **Full Payment Gateway Integration:** Implementing a complete, secure payment processing module.
- **Social Sharing & Gamification:** Adding features like in-app chat and loyalty badges.
- **Advanced Analytics for Organizers:** Integrating detailed demographic data and ticket sales prediction tools.

REFERENCES

1. Meta Open Source. "React - A JavaScript library for building user interfaces."

Available: <https://react.dev/>.

OpenJS Foundation. "Express - Fast, unopinionated, minimalist web framework for Node.js."

Available: <https://expressjs.com/>.

2. The PostgreSQL Global Development Group. "PostgreSQL: The World's Most Advanced Open Source Relational Database." Available:

<https://www.postgresql.org/>.

Lucidchart. "What is a Data Flow Diagram." Available:

<https://www.lucidchart.com/pages/data-flow-diagram>.

APPENDIX: SOURCE CODE

Backend Connection (mongodb.ts)

```
import mongoose from "mongoose"

const MONGODB_URI = process.env.MONGODB_URI

if (!MONGODB_URI) {

  throw new Error("Please define the MONGODB_URI environment variable inside
.env.local")

}

let cached = global.mongoose

if (!cached) {

  cached = global.mongoose = { conn: null, promise: null }

  if (!cached.promise) {

    const opts = {

      MONGODB_URI,
      opts
    }

    cached.promise = mongoose.connect(MONGODB_URI!, opts).then((mongoose) => {

      return mongoose

    })

    try {

      cached.conn = await cached.promise

    } catch (e) {

      cached.promise = null

      throw e

    }

    return cached.conn

  }

  export default dbConnect
```

Frontend Component (EventList.jsx)

```
import React, { useEffect, useState } from "react";

const EventList = () => {

  const [events, setEvents] = useState([]); const getEvents = async () => {

    try {

      const response = await fetch("http://localhost:5000/events");

      const jsonData = await response.json();

      setEvents(jsonData); } catch (err) {
```

17 | Page

```
      console.error(err.message);

    } };
```

```
useEffect(() => {

  getEvents();

}, []);

return (

  <div className="grid grid-cols-3 gap-4">

    {events.map(event => (

      <div key={event.event_id} className="card">

        <h3>{event.title}</h3>

        <p>{event.description}</p>

      </div>

    ))} </div> );

});

export default EventList;
```

18 | Page